CGNS Proposal for Extension 0051

Implicit FaceCenter Indexing for Unstructured Zones with Explicitly Defined Face Sections

Version 2.0

July 3, 2026

CPEX#	0051
Scope	Drop the `PointList`/`PointRange` requirement for `FaceCenter` data on unstructured zones whose face sections (`NGON_n` or classical 2-D) form one contiguous element-numbering block
Champion	M. Scot Breitenfeld
Organization	The HDF Group
E-mail	brtnfld@hdfgroup.org
GitHub Issue	TBD (implementation gaps tracked as #964, #965)
Reference Impl.	TBD
Target Release	CGNS v5.0.0 (major release)
Date First Posted	TBD (not yet posted; internal draft)
SIDS Status	proposed
Filemap Status	proposed
MLL Status	proposed

Contents

1 Abstract

SIDS Section 7.7 (`FlowSolution_t`), Note 5, states that for unstructured zones `GridLocation` is restricted to `Vertex` or `CellCenter` unless a `PointList` or `PointRange` is also present. The same restriction appears, near word-for-word, in Section 12.4 (`DiscreteData_t`). This forces every `FaceCenter` solution on an unstructured zone to carry an explicit point set — even when the data spans every face the zone defines, in which case the point set is the identity permutation and conveys no information beyond what is already implied by the zone's own element numbering.

The key observation is that whenever a face exists as an addressable entity in an unstructured zone at all, it exists as an *element* of a face-type `Elements_t` section — `NGON_n` or a classical 2-D type (`TRI_3`, `QUAD_4`, ...) — and therefore already carries a unique global element number (SIDS §7.3). A face that is defined by no section (e.g. an interior face of a purely `HEXA_8` mesh) has no element number, and *cannot* be addressed by an explicit `PointList` today either: the point set requirement never made such faces reachable, it only re-stated numbering the file already contains for the faces that are reachable. "All faces," for the purposes of face-centered data, has therefore always meant "all faces the zone explicitly defines": all faces of the grid when `NGON_n` is used (SIDS §7.3 guarantees `NGON_n` specifies them all), and exactly the section-defined — typically boundary — faces otherwise.

This CPEX proposes a precisely scoped relaxation built on that observation: when an unstructured zone has *complete face-section coverage* (defined in Section 3.2 — its face-type sections form a single contiguous block of the zone's global element numbering), `GridLocation = FaceCenter` data may be written and read *without* a `PointList`/`PointRange`, indexed directly against that block, exactly as `CellCenter` is indexed against the zone's `CellDimension`-order elements. The relaxation is strictly additive: files using the existing identity-point-set workaround remain fully conformant. Version 1.x of this proposal was scoped to `NGON_n` zones only; it was broadened to classical face sections in response to pre-submission review feedback (see Section 12), which observed that the identity point set is exactly as redundant for a boundary-patch `QUAD_4` block as it is for an `NGON_n` block, for the same element-numbering reasons.

While researching the current behavior, two independent implementation gaps were found that this CPEX's reference implementation also addresses: `cg_sol_write` cannot create a `FaceCenter`-located solution directly (Section 2.5.1), and `cgnscheck` never reads a `FlowSolution_t`'s `PointList`/`PointRange` when validating data-array sizes (Section 2.5.2), so today's identity-point-set files are not actually verifiable by the reference conformance tool.

2 Motivation and Rationale

2.1 Problem Statement

A common CFD data-storage need is a quantity defined on *every* face of an unstructured mesh (e.g., face-normal velocities, surface fluxes on all element boundaries). For a zone built from `NFACE_n`/`NGON_n` polyhedral connectivity, the following tree is representative:

```
fluid_zone Zone_t [[27  8  0]]
+---ZoneType ZoneType_t "Unstructured"
+---NGonElements Elements_t [22  0]
|   +---ElementRange IndexRange_t [ 1 36]
|   :
+---NFaceElements Elements_t [23  0]
|   +---ElementRange IndexRange_t [37 44]
|   :
+---CellData FlowSolution_t
|   +---GridLocation GridLocation_t "CellCenter"
|   +---Pressure DataArray_t   /* 8 values -- no PointList/PointRange */
+---FaceData FlowSolution_t
    +---GridLocation GridLocation_t "FaceCenter"
    +---NormalX DataArray_t     /* 36 values */
    :
    +---PointRange IndexRange_t [[1, 36]]
        ^^^^ identity range -- required by Note 5, conveys no new
            information
```

`CellData` needs no point set: `GridLocation = CellCenter` alone is sufficient, because the zone's cells are already uniquely and completely numbered by its volume `Elements_t` sections. `FaceData`, however, is required by SIDS Note 5 to carry a `PointRange`, even though `[1,36]` is exactly the `NGonElements` section's own `ElementRange` — information the reader already has.

The same redundancy occurs, for the same reason, on classical (non-polyhedral) meshes. Consider a regular 5×5×5 `HEXA_8` zone whose boundary faces are exposed as a `QUAD_4` section — the standard layout for attaching `BC_t` definitions:

```
fluid_zone Zone_t [[216  125  0]]
+---ZoneType ZoneType_t "Unstructured"
+---Hexas Elements_t [17  0]          /* HEXA_8 */
|   +---ElementRange IndexRange_t [  1 125]
|   :
+---BoundaryQuads Elements_t [7  0]    /* QUAD_4 */
|   +---ElementRange IndexRange_t [126 275]
|   :
+---CellData FlowSolution_t
|   +---GridLocation GridLocation_t "CellCenter"
|   +---Pressure DataArray_t   /* 125 values -- no PointList/PointRange */
+---FaceData FlowSolution_t
    +---GridLocation GridLocation_t "FaceCenter"
    +---NormalX DataArray_t     /* 150 values -- every defined face */
    :
```

```
    +---PointRange IndexRange_t [[126, 275]]
        ^^^^ identity range again -- the QUAD_4 section's own ElementRange
```

Here the 150-entry solution covers every face the zone *defines*. The zone's 300 interior faces are not addressable at all — they have no element numbers, so no `PointList` could reference them today either; the point-set requirement has never made them reachable. For the faces that *are* reachable, `[126,275]` is again exactly the face section's own `ElementRange`, restated by hand. Two consequences follow in both the polyhedral and the classical case:

1. **Confusion.** Users and tool authors reasonably ask why an identity point set is mandatory for one `GridLocation` and forbidden (well, unnecessary) for another, when both are governed by the same element-numbering machinery.
2. **Detection cost.** Any code path that wants to special-case "this solution spans every face" — to avoid gather/scatter overhead in a parallel read, for instance — must first read the point set and compare it against the face count, on every zone, for every solution. This is a small but repeated and entirely avoidable per-solution check.

2.2 Origin of the Restriction

Note 5 was introduced by CPEX 0031 ("revamped usage of `PointList` and `PointRange`"), incorporated into SIDS v3.1. The SIDS gives its own rationale immediately following the notes list in Section 7.7:

> *"For unstructured grids, the value of `GridLocation` alone specifies location and indexing of flow solution data only for vertex and cell-centered data. The reason for this is that element-based grid connectivity provided in the `Elements_t` data structures explicitly indexes only vertices and cells. For data stored at alternate grid locations (e.g., edges), additional connectivity information is needed. This is provided by the optional fields `PointRange` and `PointList`..."*

This is a correct and still-necessary rule for the *general* case: an unstructured zone is only ever *required* to number its vertices and its `CellDimension`-order elements. A zone built purely from `TETRA_4/PYRA_5/PENTA_6/HEXA_8/ MIXED` sections, with no face-type `Elements_t` section at all, has no face numbering whatsoever — `FaceCenter` is genuinely meaningless there without an explicit point set.

However, the rationale's premise — that `Elements_t` explicitly indexes "only vertices and cells" — is simply not true of a zone that carries face-type sections. Any `TRI_3/QUAD_4` boundary section, and any `NGON_n` section, explicitly indexes faces: that is what an element section *is*. Notably, SIDS v3.1 is also the version in which `NFACE_n` was formalized and the `NGON_n` description was substantially revised (per the same changelog entry). The changes were made in the same release but were not

cross-referenced: Note 5 was written for, and still correctly describes, the zone-with-no-face-sections case, but it was never revisited to carve out the cases — `NGON_n` or classical — where a usable face numbering *does* exist in the file.

2.3 Why Explicit Face Sections Change the Picture

SIDS Section 7.3 (`Elements_t`) already gives every element of every section — faces included — a unique global number:

> *"The elements are indexed with a global numbering system, starting at 1, for all element sections under a given `Zone_t` data structure. The global numbering insures that each element, whether it's a cell, a face, or an edge, is uniquely identified by its number. They are also listed as a continuous list of element numbers within any single element section."*

This guarantee is type-agnostic: a `QUAD_4` boundary section's faces are exactly as uniquely and contiguously numbered as an `NGON_n` section's. The only thing that varies by element type is *which* faces get numbers. For `NGON_n`, §7.3 additionally guarantees completeness over the whole grid:

> *"Arbitrary polyhedral elements may be defined using the `NGON_n` and `NFACE_n` element types. **The `NGON_n` element type is used to specify all the faces in the grid**, and the `NFACE_n` element type is then used to define the polyhedral elements as a collection of these faces."*

For classical 2-D sections there is no such completeness statement — they number only the faces they define, conventionally the boundary. But completeness over the whole grid was never what the point-set mechanism delivered either: an explicit `PointList` can only reference faces that have element numbers, so on a classical mesh it, too, can only ever address the section-defined faces (Section 2.1's second example). The set of faces reachable by `FaceCenter` data is *identical* with or without a point set — it is always "the faces the zone's sections define." What the point set adds is only a restatement of *which contiguous element-ID block* that is, and whenever the zone's face sections form a single contiguous block, that restatement is derivable from the `ElementRange`s the reader already has. This is structurally identical to how a zone's volume `Elements_t` sections already give `CellCenter` an implicit, unambiguous numbering. The `NGON_n` completeness guarantee remains meaningful, but its role is interpretive, not structural: it tells the reader the block covers *all* faces of the grid, rather than the boundary subset; the indexing machinery is the same in both cases.

2.4 Precedent: `CellCenter` Already Works This Way

SIDS Section 7.7 defines the `DataSize[]` structure function, which is the authoritative statement of how `FlowSolution_t` array sizes are derived. Quoted verbatim from

the current SIDS source (non-`Rind` branches; the `Rind`-present branch follows the same pattern):

```
if (PointRange/PointList is present) then
  {
  DataSize[] = ListLength[] ;
  }
else if (Rind is absent) then
  {
  if (GridLocation = Vertex) or (GridLocation is absent)
    {
    DataSize[] = VertexSize[] ;
    }
  else if (GridLocation = CellCenter) then
    {
    DataSize[] = CellSize[] ;
    }
  }
```

The `CellCenter` branch already relies *entirely* on implicit element-based sizing: no `PointList`/`PointRange` is required there, because `CellSize[]` is already well-defined from the zone's `CellDimension`-order `Elements_t` sections. This CPEX asks for nothing more than extending that same, already-accepted pattern to `FaceCenter` wherever the zone's face-type sections provide an equally unambiguous numbering — a single contiguous element-ID block (Section 3.2). See Section 3.5 for the precise proposed addition to `DataSize[]`.

2.5 Existing Implementation Gaps

Two related gaps in the reference implementation were found while preparing this proposal. Neither is created by this CPEX, but both must be fixed as part of its reference implementation, since they sit directly in the code paths this CPEX touches.

2.5.1 `cg_sol_write` rejects plain `FaceCenter`

`cg_sol_write` (`cgnslib.c`) validates its `location` argument against an explicit allow-list:

```
if (location != CGNS_ENUMV(Vertex) &&
    location != CGNS_ENUMV(CellCenter) &&
    location != CGNS_ENUMV(IFaceCenter) &&
    location != CGNS_ENUMV(JFaceCenter) &&
    location != CGNS_ENUMV(KFaceCenter)) {
    cgi_error("Given grid location not supported for FlowSolution_t");
    return CG_ERROR;
}
```

Plain `FaceCenter` — the only location an unstructured zone can use for face data —

is not in this list. This does *not* block the *explicit*-point-set form in Example 5.1: `cg_sol_ptset_write()` already creates a `FaceCenter` solution with an accompanying `PointList`/`PointRange` in a single call today, because it validates via `cgi_check_location` directly and, internally, works around `cg_sol_write`'s allow-list by calling it with `location = Vertex` and overwriting `sol->location` afterward — itself telling evidence that the allow-list is redundant with `cgi_check_location` rather than an independent rule (see Section 3.6.2). The gap that *does* block today is the *point-set-free* form this CPEX introduces: `cg_sol_ptset_write()` requires `npnts > 0`, so it cannot be used to create a bare `FaceCenter` solution with no point set at all, and `cg_sol_write` is the only entry point for that shape — which is exactly where the allow-list rejects it. Today, the only way to create a point-set-free `FaceCenter` solution is the two-step sequence `cg_sol_write(..., CellCenter, ...)` followed by `cg_gridlocation_write(..., FaceCenter)`, which happens to work only because `cg_gridlocation_write` performs its own, separate validation (`cgi_check_location`) that *does* permit `FaceCenter` for `CellDimension` ≥ 3. This is a latent usability bug independent of Note 5, and it directly affects every user who wants to write a point-set-free `FaceCenter` solution today (e.g. the identity-`PointRange` workaround plus later `cg_gridlocation_write` override, or any future use of this CPEX's implicit form), whether or not this CPEX is adopted. Tracked as CGNS/CGNS #964.

2.5.2 `cgnscheck` never reads `FlowSolution_t`'s point set

`check_solution()` (`cgnscheck.c`) computes the expected data-array size via `get_data_size()`, which contains:

```
if (z->type == CGNS_ENUMV(Unstructured)) {
    error ("grid location %s not valid for unstructured zone",
        cg_GridLocationName (location));
    return 0;
}
```

This branch is checked *before* `get_data_size()` ever looks at `GridLocation = FaceCenter`, so it fires for *any* `GridLocation` other than `Vertex`/`CellCenter` on an unstructured zone — including `FaceCenter` with a fully populated, conformant `PointRange`. `check_solution()` never calls `cg_ptset_info`/`cg_ptset_read` at all, so today's required identity-point-set workaround (Section 2.1) is not merely *unvalidated* by the reference conformance checker — it is *actively misreported*. `error()` is non-fatal but not silent: it prints a counted `ERROR:` line and increments `cgnscheck`'s error tally for every `FaceCenter` solution on every unstructured zone in the file, today, for files that are fully conformant under the *current* SIDS. This is a stronger motivation than mere non-validation: it is a false-positive in the reference tool that this CPEX's fix also happens to resolve. This CPEX's reference implementation fixes `get_data_size()` to honor `ListLength[]` from an explicit point set (already

required by the current SIDS) in addition to the new implicit face-section sizing this CPEX adds. Tracked as CGNS/CGNS #965.

3 Detailed Description

3.1 Design Overview

The proposal has four parts:

1. A formal SIDS definition of *complete face-section coverage* (Section 3.2), built entirely from existing SIDS guarantees — no new node types or structural elements are introduced.
2. A redline amendment to SIDS Note 5 in `FlowSolution_t` (§7.7) and the matching clause in `DiscreteData_t` (§12.4), and a corresponding addition to the `DataSize[]` structure function (Section 3.5).
3. MLL changes (Section 3.6) to compute and validate the implicit face count, expose it to readers via a new public accessor (Section 3.6.1), and fix the two unrelated implementation gaps from Section 2.5 (as standalone, independently backportable patches — see Section 11).
4. A test plan (Section 8) covering the coverage predicate, the new sizing rule, and the write-ordering constraint this CPEX introduces (Section 4).

3.2 Definition: Complete Face-Section Coverage

Call an `Elements_t` section of a `CellDimension`-3 unstructured zone a **face section** if its element type is `NGON_n` or a classical 2-D type (`TRI_3`, `TRI_6`, `QUAD_4`, `QUAD_8`, `QUAD_9`, or a `MIXED` section containing only such types). The zone's **face-indexing sections** are then determined by a two-tier precedence rule:

T1. If Z contains one or more `NGON_n` sections, the face-indexing sections are the `NGON_n` sections *only*. SIDS §7.3 states that `NGON_n` "is used to specify all the faces in the grid," so when present it is authoritative; classical face sections coexisting with it (typically boundary-patch duplicates kept for `BC_t`/tool compatibility) are geometric restatements of faces the `NGON_n` block already numbers, and including them would double-count those faces.

T2. Otherwise, the face-indexing sections are all of Z's classical face sections.

An unstructured zone Z has **complete face-section coverage** if and only if:

1. Z has at least one face-indexing section; and
2. the union of the `ElementRange`s of the face-indexing sections forms a single contiguous block $[E_{\text{start}}, E_{\text{end}}]$ of the zone's global element numbering (trivially true

for one section; for multiple sections, their ranges must partition $[E_{\text{start}}, E_{\text{end}}]$ with no gaps, and no element index inside that block may belong to any section that is not one of the face-indexing sections).

When these conditions hold, the face count is $N_{\text{face}} = E_{\text{end}} - E_{\text{start}} + 1$, and the **implicit `FaceCenter` indexing rule** is: the j-th entry (1-based) of the `DataArray_t` maps to global element index

$$E_{\text{start}} + j - 1, \qquad 1 \le j \le N_{\text{face}}.$$

Three points about what this rule does and does not mean:

- **"Complete" means complete over the defined faces.** Under tier T1 the block covers every face of the grid, by `NGON_n`'s own SIDS guarantee. Under tier T2 it covers every face the zone *explicitly defines* — conventionally the boundary faces. Interior faces of a classical mesh have no element numbers and are equally unreachable by an explicit `PointList` today (Section 2.3); this CPEX neither shrinks nor grows the set of addressable faces, it only removes the redundant restatement of their numbering.
- **The rule refers only to the block.** Sections outside $[E_{\text{start}}, E_{\text{end}}]$ — volume sections, and (under T1) classical boundary-patch sections — do not disqualify Z and do not change what the rule means; they simply are not part of the indexing block. Condition 2 alone guarantees the block is unambiguous regardless of what else exists in the zone.
- **Ambiguity is structurally excluded.** A reader never guesses which section a point-set-free `FaceCenter` array belongs to: the two-tier rule plus the contiguity condition yield either exactly one well-defined block or a determination that the zone does not qualify (in which case an explicit point set remains required, exactly as today). In particular, a zone with several same-typed boundary patches qualifies only if their ranges are mutually contiguous — the array then spans all of them in global element order — and never maps to any single patch by a size coincidence.

This CPEX does not extend to `EdgeCenter`. On `CellDimension`-2 zones `FaceCenter` itself is not a valid location (SIDS §7.7's location table), and `EdgeCenter` data on 2-D or 3-D zones raises the analogous question for `BAR_2`-type sections; the machinery defined here would transfer directly, but that extension is left to a future CPEX rather than folded into this one (Section 3.7).

3.3 Revised SIDS Text: `FlowSolution_t` (§7.7, Note 5)

Current text:

> *For unstructured zones GridLocation options are limited to Vertex or CellCenter, unless one of PointList or PointRange is present.*

Proposed text:

> For unstructured zones GridLocation options are limited to Vertex or CellCenter, unless one of PointList or PointRange is present, or unless GridLocation is FaceCenter and the zone has complete face-section coverage, in which case indexing follows the zone's face-section element numbering (§7.3) directly.

3.4 Revised SIDS Text: DiscreteData_t (§12.4)

The matching clause in DiscreteData_t is amended the same way, for consistency — both notes were introduced together by CPEX 0031 and have always read the same way, modulo word order ("PointRange or PointList" here vs. "PointList or PointRange" in §7.7):

Current text:

> *For unstructured zones GridLocation options are limited to Vertex or CellCenter, unless one of PointRange or PointList is present.*

Proposed text:

> For unstructured zones GridLocation options are limited to Vertex or CellCenter, unless one of PointRange or PointList is present, or unless GridLocation is FaceCenter and the zone has complete face-section coverage, in which case indexing follows the zone's face-section element numbering (§7.3) directly.

ZoneSubRegion_t and BC_t, the other two structures touched by CPEX 0031, are deliberately *not* amended here. ZoneSubRegion_t always requires an explicit region descriptor (point set, BCRegionName, or GridConnectivityRegionName) regardless of GridLocation, so Note 5 there is not a burden in the same way. BC_t does not carry this clause at all (a BC patch is definitionally a *subset*, so an explicit point set is appropriate there even for a complete-coverage zone). See Section 9.5 for discussion of whether a future CPEX should revisit ZoneSubRegion_t as well.

3.5 Revised DataSize[] Structure Function

Section 7.7's DataSize[] definition gains one new FaceCenter case inside *each* of the two existing Rind branches, in direct parallel with the existing Vertex and CellCenter cases. The explicit PointRange/PointList branch remains first and still takes precedence, so existing identity-point-set files are unaffected; and because

the new cases sit *inside* the `Rind` branches (rather than ahead of them), a `FaceCenter` solution carrying rind elements is still sized with its rind contribution, exactly as `CellCenter` is. The redline below follows the existing SIDS pseudocode style (`then` keywords, braces on their own lines) exactly as it appears in the current SIDS source, so the amendment can be applied to the standard's text verbatim; the two new cases are marked:

```
if (PointRange/PointList is present) then
  {
  DataSize[] = ListLength[] ;
  }
else if (Rind is absent) then
  {
  if (GridLocation = Vertex) or (GridLocation is absent)
    {
    DataSize[] = VertexSize[] ;
    }
  else if (GridLocation = CellCenter) then
    {
    DataSize[] = CellSize[] ;
    }
  else if (GridLocation = FaceCenter) and              /* new */
          (zone has complete face-section coverage) then
    {
    DataSize[] = FaceSize[] ;
    }
  }
else if (Rind is present) then
  {
  if (GridLocation = Vertex) or (GridLocation is absent) then
    {
    DataSize[] = VertexSize[] + [a + b,...] ;
    }
  else if (GridLocation = CellCenter)
    {
    DataSize[] = CellSize[] + [a + b,...] ;
    }
  else if (GridLocation = FaceCenter) and              /* new */
          (zone has complete face-section coverage) then
    {
    DataSize[] = FaceSize[] + [a + b,...] ;
    }
  }
```

Here `FaceSize[]` is defined as $E_{\text{end}} - E_{\text{start}} + 1$ for the zone's contiguous face-indexing block (Section 3.2), and `RindPlanes = [a,b,...]` as in the existing definition. The name `FaceSize[]`, rather than `NFace[]`, is chosen to avoid confusion with `NFACE_n` cell counts.

Note that §12.4 requires no matching `DataSize[]` amendment: the SIDS defines `DiscreteData_t`'s `DataSize[]` by reference — "It is identical to the function `DataSize[]` defined for `FlowSolution_t`" — so the change above propagates to `DiscreteData_t` automatically. Only the §12.4 *Note* (Section 3.4) needs its own

redline.

3.6 MLL Changes

3.6.1 New internal helper

```c
/* cgns_internals.c */
int cgi_zone_face_range(cgns_zone *zone,
                        cgsize_t *start, cgsize_t *end);
    /* Returns CG_OK and the contiguous [start,end] face-indexing
     * block if the zone has complete face-section coverage per the
     * definition in this document's Detailed Description section,
     * applying the two-tier rule: NGON_n sections define the block
     * when any are present; otherwise all classical 2-D face
     * sections do. Returns CG_NODE_NOT_FOUND if the zone has no
     * face-type sections at all; returns CG_ERROR (with cgi_error()
     * detail) if face-type sections exist but the applicable tier's
     * union is not one contiguous block. The cgi_error() message
     * names the zone and, for the gap case, the interrupting section
     * (e.g. "zone 'fluid_zone': face sections [1,20] and [31,50]
     * are not contiguous (elements 21-30 belong to section 'Hexas'"),
     * so the failure is diagnosable without re-deriving element
     * ranges by hand. Rejects (CG_ERROR) any ElementRange with
     * start < 1 or end < start, and any face count that would
     * overflow cgsize_t arithmetic in the caller's size computation,
     * before returning -- both ranges are read from the file and
     * must not be trusted blindly. */
```

This helper is used by `cg_sol_write`/`cg_field_write` (and their `cg_discrete_*` counterparts) whenever `GridLocation = FaceCenter` is requested on an unstructured zone with no accompanying point set, to (a) confirm the relaxation applies and (b) compute the expected `DataArray_t` size for validation on write, exactly as `CellSize[]` is already used for `CellCenter`. Coverage is a property of the zone's `Elements_t` section layout, already resident in memory as `zone->section[]`; the reference implementation computes it once per zone (caching the result on `cgns_zone`) rather than re-scanning the section list on every `cg_field_write` call to a multi-field `FaceCenter` solution, and invalidates the cached result on `cg_section_write`/`cg_section_partial_write`.

A thin public wrapper is also added, so that reader-side code (as in Example 5.3) does not have to reimplement the coverage predicate independently for every downstream tool:

```c
/* cgnslib.c; public API */
int cg_zone_face_range(int fn, int B, int Z,
                       cgsize_t *start, cgsize_t *end);
    /* Thin wrapper over cgi_zone_face_range() for read-side
     * consumers. Same return-value contract. */
```

3.6.2 Fix: `cg_sol_write` allow-list

Rather than add `FaceCenter` as one more literal in `cg_sol_write`'s local allow-list, the reference implementation *removes* that allow-list and delegates to `cgi_check_location()` — the same dimension- and zone-type-aware check that `cg_gridlocation_write` and `cg_sol_ptset_write` already trust, and the reason those two entry points already accept `FaceCenter` correctly today while `cg_sol_write` does not. `cg_sol_write`'s own literal allow-list and its separate, hand-written `[IJK]FaceCenter`/`Structured`-only check are two independent copies of a rule `cgi_check_location` already enforces; keeping both is exactly the kind of duplication that let them drift apart and produced this bug in the first place. Because `cgi_check_location` needs the zone's `CellDimension` and `ZoneType_t`, the check must move to *after* the zone lookup (`cg_sol_write` currently validates `location` before `cgi_get_file`/`cgi_get_zone` are called):

```c
/* was: a literal allow-list checked before the zone lookup, plus a
 * separate structured-only [IJK]FaceCenter check further down. Both
 * are replaced by one delegated call, placed after the zone lookup: */
zone = cgi_get_zone(cg, B, Z);
if (zone == 0) return CG_ERROR;
if (cgi_check_location(cg->base[B-1].cell_dim, zone->type, location))
    return CG_ERROR;   /* cgi_check_location has already set cgi_error()
        */
```

This is strictly additive in effect: every `location` value previously accepted by the old allow-list is still accepted (each is one of `cgi_check_location`'s `CG_OK` cases), plain `FaceCenter` is now accepted for `CellDimension` $\geq$ `3` regardless of zone type (matching what `cg_gridlocation_write` already permits today), and `[IJK]FaceCenter` on a non-`Structured` zone is still rejected, with the same error message, by `cgi_check_location` itself. This fix is independent of the rest of this CPEX (it does not by itself relax Note 5) but is required in order for Example 5.2's single-call write to work; without it, applications must still fall back to the two-step `cg_sol_write` + `cg_gridlocation_write` sequence for the point-set-free form. Being a narrowly-scoped, independently testable bug fix with no dependency on the face-section-coverage logic, it can land as its own patch against the current release line without waiting on the rest of this CPEX; see the note in Section 11.

3.6.3 Fix: `cgnscheck` point-set-aware sizing

`get_data_size()` (Section 2.5.2) is updated to: (1) read a `PointList`/`PointRange` under `FlowSolution_t`/`DiscreteData_t`, if present, and use its `ListLength` as the expected size (this branch is required by the *current* SIDS and was simply never implemented); and (2) when no point set is present and `GridLocation = FaceCenter` on an unstructured zone, call `cgi_zone_face_range()` and use the returned count. Only when neither applies does `get_data_size()` retain its current "grid location not valid for unstructured zone" error.

3.6.4 No change to on-disk representation

No new node type, attribute, or data layout is introduced. A conformant file under this CPEX is a `FlowSolution_t` (or `DiscreteData_t`) node with `GridLocation = FaceCenter` and *no* `PointList`/`PointRange` child, on a zone with complete face-section coverage. Because this is a strict superset of what Note 5 currently forbids, this CPEX changes only the *normative interpretation* of an existing, legal-today node shape — it does not add anything a pre-CPEX HDF5/ADF reader would fail to open at the byte level.

3.7 Scope / Non-Goals

- **Not addressed:** `EdgeCenter`. The v2.0 machinery (Section 3.2) would transfer directly to `EdgeCenter` data indexed against `BAR`-type sections — the v1.x objection ("no completeness guarantee for edges") no longer applies under the "complete over the defined elements" reading — but the extension multiplies the location/dimension test matrix and is deliberately left to a future CPEX rather than folded into this one.
- **Not addressed:** the structured-zone `GridLocation` gap raised during the same pre-submission review: 2-D structured zones have no directional edge-center locations (`IEdgeCenter`/`JEdgeCenter`) analogous to 3-D's `IFaceCenter`/`JFaceCenter`/`KFaceCenter` — `GridLocation_t` today offers only the undifferentiated `EdgeCenter`. This is a real asymmetry in the enumeration, but it concerns structured zones and a new enumeration value, orthogonal to this CPEX's unstructured-zone/no-new-enumerations scope; it is recorded here as a candidate companion CPEX (Section 10).
- **Not addressed:** `ZoneSubRegion_t`, `BC_t`. See Section 9.5.
- **Not required:** no change to `NFACE_n`'s own definition, or to how `CellCenter` is already handled; both are unaffected and serve as the existing precedent this CPEX generalizes.

4 Backward Compatibility

- **Existing files:** Fully compatible. Files using the current identity-point-set workaround remain valid; the explicit `PointRange`/`PointList` branch of the revised `DataSize[]` function still takes precedence over the new implicit branch, so nothing about their interpretation changes.
- **Existing readers (pre-CPEX-0051).** A pre-CPEX reader that encounters a `FaceCenter FlowSolution_t` with no point set on a qualifying zone will, under the *old* SIDS text, consider the file non-conformant (Note 5 as currently written requires the point set), and a naive reader may simply misinterpret the array size instead of erroring cleanly. This is the one real compatibility wrinkle, and it deserves emphasis: writers that adopt this CPEX are producing files that are conformant under the *new* SIDS text but not the old one, so **a writer must not omit the point set unless it knows its downstream tool ecosystem supports this CPEX**. CGNS versions its SIDS compliance via `CGNSLibraryVersion_t` (and, if CPEX 0049 is adopted, `CGNSMinRequiredVersion_t`). If CPEX 0049 is adopted, this CPEX **must** be registered in its Feature-Version Registry (CPEX 0049, "Feature-Version Registry" section) so that AUTO write mode raises a file's minimum required version whenever the implicit `FaceCenter` form is used, and so that version-aware readers reject or warn appropriately rather than silently misreading array sizes. If CPEX 0049 is not adopted, this CPEX should instead mandate a plain `CGNSLibraryVersion_t` bump on write, as the minimum viable version-gating mechanism.

  Why a major release. This CPEX is targeted at CGNS **v5.0.0**, the next major release, rather than a point release on the v4.x line. This is not merely a labeling choice: under CPEX 0049's revised `cg_open` logic, a library whose major version is older than the file's major version is rejected immediately, before any feature-compatibility check runs (CPEX 0049, "Modified `cg_open` Behavior," major-version safety guard). If this CPEX lands in v5.0.0 alongside CPEX 0049, a v4.x reader opening a v5.0-written file is hard-rejected at that guard and never reaches the point where it would have to interpret a point-set-free `FaceCenter` solution at all, eliminating the "silently misreads the data" failure mode entirely for that population of readers. This is a stronger guarantee than a minor-version bump could offer under the same mechanism, since CPEX 0049 only hard-rejects on a major-version mismatch; a same-major, higher-minor file is instead routed through the feature-aware check, which depends on this CPEX's Feature-Version Registry entry being present and correctly detected.
- **Existing writers:** Unaffected. Codes that do not use the new implicit form continue to write the explicit point set exactly as before; nothing is deprecated or removed. *Today*, this ordering independence is unconditional: `cg_field_write`

branches on whether a point set is already attached to the solution at the moment *it* is called (`sol->ptset == NULL`), and writing a `FaceCenter` field before any point set exists already fails on an unstructured zone (no implicit sizing rule exists yet), so no existing writer can depend on a field-before-point-set ordering. This CPEX changes that: once implicit sizing exists, a solution written field-first would size against the face-section block count, and a point set added *afterward* would silently reinterpret an already-on-disk array written to a different, incompatible size. The reference implementation must therefore forbid attaching a `PointList`/`PointRange` to a `FlowSolution_t`/ `DiscreteData_t` node that already has one or more `DataArray_t` children written under implicit sizing — returning `CG_ERROR` from the point-set write call in that case — rather than silently permitting the reinterpretation. This is a new ordering constraint introduced by this CPEX, not a preexisting one, and should be covered by its own test case (Section 8).

- **ABI:** No structure layout changes are required beyond the addition of the internal `cgi_zone_face_range()` helper and its public wrapper `cg_zone_face_range()` (Section 3.6.1). The public API changes are limited to widening `cg_sol_write`'s accepted `location` values (already a bug fix independent of this CPEX) and adding the new read-side wrapper, and are purely additive in effect (previously-rejected input is now accepted; no previously-accepted input is rejected; the wrapper is an optional convenience with no existing caller). **No new public error codes are introduced.** A write requesting implicit `FaceCenter` sizing on a zone that lacks complete face-section coverage returns the existing `CG_ERROR` (with a `cgi_error()` message identifying the missing coverage), exactly as an out-of-range `GridLocation` does today; `cg_zone_face_range()` returns the same `CG_OK`/`CG_NODE_NOT_FOUND`/`CG_ERROR` contract as every other existing `cg_*_info`-style read function, so no new return-code semantics are introduced at the public boundary either.
- **Parallel I/O (`cgp_*`):** Unaffected. The relaxation changes only whether a point-set child node must exist; it does not change the collective/independent write model for `DataArray_t` contents.

5 Examples

5.1 Example 1: Before — Identity `PointRange` (current, still valid)

```
int S, F;
cg_sol_write(fn, B, Z, "CellData", CGNS_ENUMV(CellCenter), &S);
cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble), "Pressure",
               pressure, &F);

/* FaceCenter WITH an explicit point set already works today in one
 * call via cg_sol_ptset_write -- the gap this CPEX's reference
 * implementation fixes (see Existing Implementation Gaps, above) is
 * specifically the point-set-FREE form below, not this one. The
 * identity PointRange must still be kept in sync with NGonElements'
 * ElementRange by hand. */
cgsize_t range[2] = {1, 36};
cg_sol_ptset_write(fn, B, Z, "FaceData", CGNS_ENUMV(FaceCenter),
                   CGNS_ENUMV(PointRange), 2, range, &S);
cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble), "NormalX",
               normal_x, &F);
```

5.2 Example 2: After — Implicit `FaceCenter` (this CPEX)

```
int S, F;

/* FaceCenter now accepted directly by cg_sol_write (fix described in
 * this CPEX's MLL Changes section); no PointRange needed -- the
 * zone's NGonElements section already defines the face numbering. */
cg_sol_write(fn, B, Z, "FaceData", CGNS_ENUMV(FaceCenter), &S);
cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble), "NormalX",
               normal_x, &F);   /* size validated against the zone's
                                 * face-section block count */
```

Resulting tree (compare to the first example in Section 2.1):

```
fluid_zone Zone_t [[27  8  0]]
+---ZoneType ZoneType_t "Unstructured"
+---NGonElements Elements_t [22  0]
|   +---ElementRange IndexRange_t [ 1 36]
+---NFaceElements Elements_t [23  0]
|   +---ElementRange IndexRange_t [37 44]
+---CellData FlowSolution_t
|   +---GridLocation GridLocation_t "CellCenter"
|   +---Pressure DataArray_t
+---FaceData FlowSolution_t
    +---GridLocation GridLocation_t "FaceCenter"
    +---NormalX DataArray_t     /* 36 values; indexed 1..36 against
                                    NGonElements, no PointRange node */
```

The MLL's Fortran bindings (`cg_sol_write_f`, `cg_field_write_f`) forward the same `location` argument through the same validation as their C counterparts, so both the `cg_sol_write` allow-list fix and this CPEX's implicit-sizing behavior are directly visible to Fortran callers, who make up a substantial share of CGNS writers. The Fortran mirror of Example 5.2:

```
integer :: S, F, ier

! FaceCenter now accepted directly by cg_sol_write_f; no PointRange
! needed -- the zone's NGonElements section defines the numbering.
call cg_sol_write_f(fn, B, Z, 'FaceData', FaceCenter, S, ier)
if (ier .ne. CG_OK) call cg_error_exit_f()
call cg_field_write_f(fn, B, Z, S, RealDouble, 'NormalX', &
                      normal_x, F, ier)
if (ier .ne. CG_OK) call cg_error_exit_f()
```

5.3 Example 3: Reading — Dispatching on Coverage

```
char name[33];
CGNS_ENUMT(GridLocation_t) loc;
CGNS_ENUMT(PointSetType_t) ptype;
cgsize_t npnts, fstart, fend;

if (cg_sol_info(fn, B, Z, S, name, &loc)) return handle_error();

/* cg_sol_ptset_info addresses solution S directly and reports "no
 * point set" as CG_OK with ptype == PointSetTypeNull -- unlike the
 * cg_goto-positioned cg_ptset_info, it needs no prior cg_goto call,
 * and a real read failure is distinguishable from "point set absent". */
if (cg_sol_ptset_info(fn, B, Z, S, &ptype, &npnts)) return handle_error();
int has_ptset = (ptype != CGNS_ENUMV(PointSetTypeNull));

if (loc == CGNS_ENUMV(FaceCenter) && !has_ptset) {
    /* Node shape alone (FaceCenter, no point set) is necessary but
     * NOT sufficient: a pre-CPEX-0051 or non-conformant file can
     * present this same shape without complete face-section
     * coverage. Verify before trusting it -- this CPEX makes the
     * check cheap, not optional, which is why cg_zone_face_range
     * is public. */
    if (cg_zone_face_range(fn, B, Z, &fstart, &fend) != CG_OK) {
        /* Claims the implicit form but coverage doesn't hold:
         * non-conformant file. Fail safely rather than guessing a
         * size -- this is the "silently misreads the data" failure
         * mode Section 4 warns against. */
        return handle_nonconformant_file(fn, B, Z, S);
    }
    /* Spans every face the zone defines: all faces of the grid on
     * an NGON_n zone (tier T1), the section-defined (typically
     * boundary) faces on a classical zone (tier T2). */
    process_all_defined_faces(fn, B, Z, S, fend - fstart + 1);
} else if (loc == CGNS_ENUMV(FaceCenter) && has_ptset) {
    /* Explicit form: a genuine subset, or a pre-CPEX-0051 file. */
    process_face_subset(fn, B, Z, S, ptype, npnts);
}
```

5.4 Example 4: Rejected — Non-Contiguous Face Sections

```c
/* Case A (polyhedral): TWO NGON_n sections separated by an
 * unrelated section, e.g. ElementRange [1,20] (NGON_n), [21,30]
 * (some other type), [31,50] (NGON_n). Condition 2 of the coverage
 * definition is violated for tier T1: the NGON_n ranges do not form
 * a single contiguous block, so there is no well-defined
 * [E_start, E_end] to index against.
 *
 * Case B (classical): QUAD_4 patches interleaved with volume
 * sections, e.g. HEXA_8 [1,100], QUAD_4 [101,150], HEXA_8
 * [151,200], QUAD_4 [201,250]. Tier T2 applies (no NGON_n), and
 * the union of the two QUAD_4 ranges is not contiguous. In both
 * cases the explicit point set is still required -- notably, this
 * is what structurally prevents a same-sized-patch ambiguity: the
 * implicit form never attaches to one patch among several by a
 * size coincidence. */
cg_sol_write(fn, B, Z, "FaceData", CGNS_ENUMV(FaceCenter), &S);
/* cg_field_write size-validation falls back to requiring an
 * explicit PointList/PointRange, and cgnscheck flags its absence. */
```

5.5 Example 5: Accepted — NGON_n Coexisting with a Boundary-Patch Section

```c
/* Zone has an NGON_n section [1,36] covering every face, PLUS a
 * separate QUAD_4 section [37,44] duplicating the boundary faces
 * for a BC_t patch -- a common pattern for tool compatibility.
 * This qualifies under tier T1: NGON_n is present, so it alone
 * defines the face-indexing block; the QUAD_4 duplicates are
 * excluded, which is what prevents the boundary faces from being
 * counted twice (N stays 36, not 44). The QUAD_4 range must merely
 * lie outside the NGON_n block, which it does. */
cg_sol_write(fn, B, Z, "FaceData", CGNS_ENUMV(FaceCenter), &S);
cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble), "NormalX",
               normal_x, &F);   /* 36 values, indexed against [1,36] */
```

5.6 Example 6: Accepted — Classical Mesh, Boundary Faces Only (new in v2.0)

The 5×5×5 `HEXA_8`/`QUAD_4` zone from Section 2.1: no `NGON_n` anywhere, one classical face section `[126,275]`.

```c
/* Tier T2: no NGON_n sections, so the face-indexing block is the
 * union of the classical face sections -- here the single QUAD_4
 * section [126,275], trivially contiguous. N_face = 150. */
cg_sol_write(fn, B, Z, "FaceData", CGNS_ENUMV(FaceCenter), &S);
cg_field_write(fn, B, Z, S, CGNS_ENUMV(RealDouble), "NormalX",
               normal_x, &F);   /* 150 values, indexed against
                                 * [126,275] -- every DEFINED face.
                                 * The 300 interior faces have no
                                 * element numbers and remain exactly
                                 * as unaddressable as they are today
                                 * with an explicit point set. */
```

Multiple boundary patches also qualify when contiguous: three `QUAD_4` sections `[126,175]`, `[176,225]`, `[226,275]` (e.g. `INFLOW/OUTFLOW/WALL`) partition `[126,275]` with no gaps, so the zone qualifies with the same $N_{\text{face}} = 150$; the implicit array spans all three patches in global element order. Per-patch data still uses an explicit point set, exactly as today.

6 SIDS Impact

- §7.7 `FlowSolution_t`, Note 5: amended (Section 3.3).
- §7.7 `FlowSolution_t`, `DataSize[]` function: amended (Section 3.5).
- §12.4 `DiscreteData_t`, equivalent Note: amended (Section 3.4). No `DataSize[]` amendment is needed in §12.4 itself: the SIDS defines `DiscreteData_t`'s `DataSize[]` by reference ("It is identical to the function `DataSize[]` defined for `FlowSolution_t`"), so the §7.7 change propagates automatically (Section 3.5).
- §7.3 `Elements_t`: unchanged. This CPEX relies entirely on the existing global-numbering guarantee (all element types) and, for the tier rule, the "`NGON_n` specifies all faces" statement already there (Section 2.3); no new text is needed in §7.3 itself.
- No new SIDS node types, enumerations, or structural fields are introduced anywhere.

7 File Mapping Impact

None. This CPEX changes only which combinations of existing nodes are considered conformant; it introduces no new HDF5/ADF on-disk structures, attributes, or data types. The ADFH and ADF file-mapping documents require no amendment.

8 Test Plan

The reference implementation's test suite must cover at least the following fixtures. Items 1–7 exercise the coverage predicate (both tiers) and the new sizing rule directly; items 8–9 pin down behavior this CPEX makes newly reachable and that would otherwise be left implicit; item 10 guards the one existing entry point this CPEX widens.

1. Single `NGON_n` section, implicit form (Example 5.2): round-trip write, read, and `cgnscheck` clean (zero errors) in both C and Fortran (Fortran mirror of Example 5.2).

2. Multiple contiguous `NGON_n` sections partitioning one `[E_start, E_end]` block: accepted, sized correctly.

3. Classical-only zone, tier T2 (Example 5.6): single `QUAD_4` boundary section on a `HEXA_8` mesh — round-trip write, read, `cgnscheck` clean, N_{face} equal to the section's `ElementSize`. Repeat with three contiguous same-typed patches partitioning the block, and with a mixed `TRI_3`+`QUAD_4` contiguous pair. *New in v2.0; no installed base exists, so these fixtures carry the highest regression value in the suite alongside item 4.*

4. Mixed zone, tier T1 precedence (Example 5.5): `NGON_n` block plus classical boundary-patch duplicates outside it — accepted, and N_{face} must equal the `NGON_n` block size alone (36, not 44, in the example's numbers), pinning down that classical duplicates are excluded and boundary faces are not double-counted.

5. Gap cases (Example 5.4, both Case A and Case B): write of a solution requesting implicit sizing is rejected with `CG_ERROR`; a file constructed to have this shape externally is flagged by `cgnscheck` and by `cg_zone_face_range()` returning `CG_ERROR` on read. Case B doubles as the same-sized-patch ambiguity guard: two disjoint equal-length `QUAD_4` patches must never be silently resolved to either patch.

6. Legacy identity-`PointRange` file (Example 5.1): still passes, unchanged size, confirming the explicit-point-set branch of `DataSize[]` retains precedence over the new implicit branch. Repeat for the classical-mesh identity file of Section 2.1 (`PointRange [126,275]`).

7. Zone with *no* face-type sections at all (pure `HEXA_8`/`MIXED`-volume): `cg_zone_face_range()` returns `CG_NODE_NOT_FOUND`; implicit `FaceCenter` write is rejected; the explicit-point-set path remains the only one, exactly as today.

8. Point-set-written-after-implicit-fields ordering: attempt to attach a `PointRange`/`PointList` to a `FlowSolution_t` that already holds fields written under implicit face-section sizing; must return `CG_ERROR` per the rule in Section 4. This behavior is otherwise undefined by the prose alone; the test is what pins it down.

9. Malformed `ElementRange` inputs to `cgi_zone_face_range()`: `start < 1`, `end < start`, and (on a 32-bit `cgsize_t` build) a range whose face count overflows subsequent size arithmetic — all three must return `CG_ERROR` before any allocation is attempted downstream (Section 3.6.1).

10. Structured zone with plain `FaceCenter` through the fixed `cg_sol_write` (Section 3.6.2): must still be accepted for `CellDimension` ≥ 3 (matching `cg_gridlocation_write`'s existing behavior) and rejected below that, pinning down the delegated-`cgi_check_location` behavior in both directions.

Note on `cgnscheck` regression risk. Once `get_data_size()` is fixed to honor an explicit point set (Section 2.5.2), files that previously produced a false-positive `ERROR:` for *any* reason unrelated to sizing will, for the first time, actually have their declared array size checked against `ListLength[]`. Any existing file whose `FlowSolution_t` array size does not genuinely match its point set's `ListLength` — previously masked by the unconditional unstructured-zone bailout — will newly and correctly fail. This is the widest legacy-system ripple in this proposal: it is a correctness fix, not a regression, but should be called out explicitly in `cgnscheck` release notes so it is not mistaken for one.

9 Design Decisions

9.1 Alternative 1: Scope to `NGON_n` Zones Only (the v1.x design)

Version 1.x of this CPEX restricted the relaxation to zones with `NGON_n` coverage, on the grounds that only `NGON_n` carries SIDS's "all the faces in the grid" guarantee, and that treating classical 2-D sections' combined `ElementSize` as "the face count of the zone" would misinterpret boundary-patch-only files. **Superseded in v2.0**, following pre-submission review. The v1.x rejection rested on reading "all faces" as "all geometric faces of the mesh" — but that is not, and never was, what face-centered data can address. An explicit `PointList` can only reference faces that carry element numbers, so on a classical mesh the reachable set is the section-defined faces regardless of whether the point set is present (Section 2.3). Once "complete" is read correctly as "complete over the faces the zone defines," the coincidental-size-match objection dissolves: the v2.0 rule (Section 3.2) never compares sizes against an external total at all; it requires the face sections' `ElementRange`s to form one structurally verifiable contiguous block, which either holds or does not. The identity point set is exactly as redundant for a boundary `QUAD_4` block as for an `NGON_n` block, and the summation/contiguity machinery is identical (multiple `NGON_n` sections already required it in v1.x).

9.2 Alternative 2: New Explicit Marker Node (e.g., `FacesComplete`)

Introduce a boolean/flag child of `Zone_t` or `Elements_t` stating explicitly that a section enumerates all faces — reviving, in spirit, the 2005-era "`ZoneElementType`" idea from the original face-based storage proposal that predated `NGON_n`/`NFACE_n` standardization. **Rejected.** The element sections already carry the needed meaning by SIDS definition (§7.3); adding a redundant marker node reintroduces exactly the kind of "assert something already implied" overhead this CPEX exists to remove, just relocated from `FlowSolution_t` to `Zone_t`.

9.3 Alternative 3 (chosen): Structural Inference from Face-Section Coverage

Infer coverage structurally from the conditions in Section 3.2, using only guarantees the SIDS already makes. No new nodes; no new trust assumptions beyond ones the standard already relies on for `CellCenter`. This is the design adopted by this CPEX.

9.4 Why `NGON_n` Takes Precedence in Mixed Zones (Tier Rule)

The one place where "sum all the face sections" would go wrong is the common compatibility pattern of Example 5.5: an `NGON_n` block covering every face *plus* classical boundary-patch sections duplicating the boundary faces for `BC_t`/tool consumption. A naive union over all face sections would count those boundary faces twice and silently change the implicit array's length and meaning for precisely the polyhedral files this CPEX originally targeted. The two-tier rule in Section 3.2 resolves this without any new markers: when `NGON_n` is present it is, by its SIDS definition, already the complete face set, so any coexisting classical sections are necessarily duplicates and are excluded from the indexing block; when no `NGON_n` exists, the classical sections are the only face definitions there are, and their union is the block. Each tier's justification comes directly from existing SIDS text, and the two tiers never overlap.

9.5 On Scope: Why Not Also Amend `ZoneSubRegion_t`?

`ZoneSubRegion_t` (§7.9) shares CPEX 0031 ancestry with `FlowSolution_t` and `DiscreteData_t`, but it always requires an explicit region descriptor by design — a subregion that happened to cover every face would be indistinguishable from simply using `FlowSolution_t`, so there is no analogous "identity point set" pain point to remove. This is left as an open question for the committee rather than folded into this proposal's scope.

10 Open Questions

1. Should `cgnscheck` *warn* (rather than silently accept) when it encounters the pre-CPEX-0051 identity-`PointRange` pattern on a zone with complete face-section coverage, to nudge writers toward the more compact implicit form? This proposal takes no position; it is a tooling policy choice, not a conformance question.
2. Should version gating use CPEX 0049's `CGNSMinRequiredVersion_t`/feature-mask mechanism, if adopted, or a plain `CGNSLibraryVersion_t` bump? This

CPEX assumes whichever mechanism is current SIDS practice at the time of adoption and does not mandate a specific one.

3. CPEX 0050 (`DofStorage_t`), Example 4, already writes a `FaceCenter` `TraceSolution` via a single `cg_sol_write` call with no accompanying point set, for a zone implied to have `NGON_n` coverage. That example is only fully conformant once both this CPEX (for the missing point set) and the `cg_sol_write` allow-list fix (Section 2.5.1) are adopted. If both CPEXes proceed, their reference implementations should be sequenced so CPEX 0051 lands first, or the dependency should be noted explicitly in CPEX 0050.

4. Should a companion CPEX add `IEdgeCenter`/`JEdgeCenter` to `GridLocation_t` for 2-D structured zones, restoring symmetry with 3-D's `IFaceCenter`/`JFaceCenter`/`KFaceCenter`? Raised during the v2.0 pre-submission review (Section 3.7); it requires a new enumeration value, which this CPEX deliberately avoids, so it is recorded here rather than folded in.

11 Summary of API Additions

The two implementation gaps from Section 2.5 (`cg_sol_write`'s allow-list, and `cgnscheck`'s missing point-set read) are pre-existing bugs, independent of the face-section-coverage logic itself, and do not need to wait on this CPEX's SIDS-level relaxation or on `v5.0.0`. They can and should land as standalone patches against the current release line as soon as they are reviewed; this CPEX's reference implementation only *depends on* them, so that Example 5.2's single-call write works end to end.

Symbol	Change	Release
`cg_sol_write`	Delegates location check to `cgi_check_location`, accepting `FaceCenter` directly (Sec. 3.6.2)	any (backportable)
`cg_discrete_write`	Same fix, mirrored	any (backportable)
`get_data_size()` (cgnscheck)	Point-set-aware sizing (Sec. 2.5.2)	any (backportable)
`cgi_zone_face_range()`	New internal helper (Sec. 3.6.1)	v5.0.0 (this CPEX)
`cg_zone_face_range()`	New public wrapper, read-side coverage check (Sec. 3.6.1)	v5.0.0 (this CPEX)

Aside from the new public `cg_zone_face_range()` wrapper, no new public enumerations, structs, or function signatures are introduced; `FaceCenter` already exists in

`GridLocation_t`.

12 Revision History

- **v1.0–1.2** — Initial proposal, scoped to zones with complete `NGON_n` face coverage only. Classical 2-D sections were explicitly excluded (then-Alternative 1, Section 9.1) on the grounds that the SIDS gives them no "all faces" guarantee. v1.2 corrected the implementation-gap descriptions and SIDS quotations against the MLL source and SIDS documentation source.
- **v2.0** — Broadened to classical face sections following pre-submission review, which made two observations this version adopts: (i) an explicit `PointList` can only ever reference faces that carry element numbers, so "complete" has always effectively meant "complete over the faces the zone defines," making the identity point set exactly as redundant for a boundary-`QUAD_4` block as for an `NGON_n` block; and (ii) the multi-section union/contiguity machinery was already required for multiple `NGON_n` sections, so extending it to classical sections adds no new mechanism. Renamed the coverage definition to *complete face-section coverage* with a two-tier `NGON_n`-precedence rule (Section 9.4), renamed the helper to `cgi_zone_face_range()`/`cg_zone_face_range()`, added the classical-mesh example (Example 5.6) and corresponding test fixtures, and recorded the reviewer's `IEdgeCenter`/`JEdgeCenter` suggestion as a candidate companion CPEX (Section 10).

13 References

1. CGNS SIDS, §7.3 `Elements_t` (element numbering, `NGON_n`/`NFACE_n` definition).
2. CGNS SIDS, §7.7 `FlowSolution_t` (Notes and `DataSize[]` function).
3. CGNS SIDS, §12.4 `DiscreteData_t`.
4. CGNS SIDS Changelog, Version 3.1 (CPEX 0031: PointList/PointRange revamp; `NGON_n`/`NFACE_n` formalization).
5. CPEX 0049, CGNS Compatibility Version Control (version-gating mechanism referenced in Section 4).
6. CPEX 0050, DOF Storage Type (Example 4 dependency noted in Open Question 3, Section 10).
7. S. Allmaras, "Face-Based Proposal" (2005-era precursor discussion of face-based unstructured storage, predating standardized `NGON_n`/`NFACE_n`).